

Modul 06: Routing & Guards

<i>Ziel:</i>	<i>Seitennavigation und Guards hinzufügen</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~60min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>05.08.2024</i>

1 Navigation erstellen

Die Todo-Liste und Erstellung einer neuen Aufgabe sollen auf eigenen Seiten getrennt werden. Dies erreichen wir mit einer kleinen Navigation, welche auf die Komponenten verweist. Im `app-routing.module.ts` müssen alle existierenden Routen registriert werden. Wir erweitern daher das Array `routes` um folgende Einträge:

```
const routes: Routes = [
  {
    path: '',
    redirectTo: '/list',
    pathMatch: 'full',
  },
  {
    path: 'list',
    component: TaskListComponent,
  },
  {
    path: 'new',
    component: TaskFormComponent,
  }
];
```

Die Startseite (also <http://localhost:4200>) leitet auf die Listenansicht weiter. Jedem Pfad wird eine Component zugeordnet. Um die Projektstruktur übersichtlich zu halten, empfiehlt es sich die Komponenten in einen Ordner "routes" zu verschieben.

Wir passen noch die Wurzelkomponente `app.component.html` an und fügen eine Liste mit Links hinzu. Zuerst modifizieren wir das Stylesheet in `app.component.css`:

```
ul { list-style: none; padding: 0; }
ul > li { display: inline-block; }
ul > li:not(:last-child)::after { content: '|'; padding: 0 .6rem; }
ul > li > a:hover { text-decoration: underline; }
```

Das HTML-Template sieht wie folgt aus:

```
<header>
  <h2>Hello, {{ title }}</h2>
  <nav>
    <ul>
      <li><a routerLink="/">Übersicht</a></li>
      <li><a routerLink="/new">Neue Aufgabe</a></li>
    </ul>
  </nav>
</header>

<main>
  <router-outlet />
</main>

<app-toast-list />
```

Das **router-outlet** ist der Platzhalter, wo die angegebenen Komponenten aus unserer Routing Konfiguration dargestellt werden. Ferner verwenden wir bei den Links nicht href für den Pfad, sondern die Direktive **routerLink** aus dem Router Module.

Hiermit haben wir bereits eine Funktionsfähige Navigation.

2 Fehlerseite hinzufügen

Wenn eine ungültige URL aufgerufen wird, soll eine eigene Fehlerseite aufgerufen werden. Wir legen hierzu eine routes/not-found Komponente an und ergänzen in `app-routing.module.ts` folgenden Eintrag:

```
{
  path: '**',
  component: NotFoundComponent,
},
```

Wenn wir jetzt in der Adressleiste eine unbekannte URL eintippen (z. B. <http://localhost:4200/xyz>), sollte jetzt die Fehlerseite angezeigt werden. Der Inhalt kann beliebig gestaltet werden.

3 Aufgabe bearbeiten

Es soll möglich sein, eine vorhandene Aufgabe zu bearbeiten. Dazu fügen wir dem Routing Modul noch folgender Eintrag ergänzt:

```
{
  path: 'edit/:id',
  component: TaskFormComponent,
},
```

Hiermit teilen wir Angular mit, dass es einen variablen Pfad gibt. Der Platzhalter `:id` enthält die Id unserer Aufgabe.

Wir erweitern die task-item Komponente und um einen weiteren Button Bearbeiten (✎) neben Löschen (🗑). Bei einem Klick soll eine Navigation auf die task-form Komponente erfolgen.

```
constructor(private taskService: TaskService, private router: Router) {}

editTask(id: string) {
  this.router.navigate(['/edit', id]);
}
```

Wir holen uns die Abhängigkeit `Router` um die `navigate`-Methode aufrufen zu können. Den Parameter können wir in der task-form Komponente mittels der `ActivatedRoute` Abhängigkeit auslesen und anschließend verarbeiten.

```
constructor(
  private taskService: TaskService,
  private toastService: ToastService,
  private router: Router,
  private route: ActivatedRoute
) {}

get id(): string | null {
  return this.route.snapshot.paramMap.get('id');
}
```

Wenn eine gültige Id übergeben wurde, soll der Button "Neue Aufgabe" durch "Speichern" ersetzt werden, welcher dann die Methode `updateTask(...)` aufruft. Außerdem soll in jedem Fall nach dem Klick zurück auf die Liste navigiert werden.

Bitte beachten, dass wir mit einem Snapshot arbeiten. Das heißt, wir bekommen nicht, wenn sich der Parameter Id ändert. Das wäre beispielsweise dann der Fall, wenn es ein Button "Nächste Aufgabe" gäbe, um innerhalb der Aufgaben zu navigieren. Um das zu lösen, müssten wir wieder mit Observables bzw. Signals arbeiten.

4 Daten der Aufgabe laden

Wir müssen die vorhandenen Daten der Aufgabe laden. Dafür implementieren wir `OnInit` aus dem Angular Lifecycle. Wir sollten in jedem Fall vermeiden, Logik im Konstruktor auszuführen. Zudem kann `OnInit` auch asynchron sein, was bei Datenzugriffen häufig notwendig ist.

```
export class TaskFormComponent implements OnInit {

  currentTask = <TaskItem>{};
  //...

  ngOnInit(): void {
    if (this.id) {
      const task = this.taskService.getTask(this.id);
      if (task) {
        this.currentTask = task;
      }
    }
  }
  //...
}
```

Der `TaskService` ist um die entsprechende Methode `getTask()` zu erweitern, falls noch nicht geschehen. Die Daten können wir mittels One-Way-Binding an die Input-Elemente binden.

```
<label for="dueDate">Fälligkeit</label>
<input id="dueDate" #dueDateRef type="date"
  [value]="editing ? (currentTask.dueDate | date : 'yyyy-MM-dd') : ''" />
```

Diese Lösung ist noch nicht optimal. Im Modul Formulare werden wir sehen, wie wir das verbessern können.

5 Nutzungsbedingungen mittels Guard abfragen

Wir wollen unsere App noch rechtlich absichern, indem wir vor der ersten Benutzung der App die Zustimmung des Benutzers einfordern. Dies können wir mit einem `CanActivate-Guard` realisieren.

```
ng generate guard guards/terms-of-use
ng g c routes/terms-of-use
```

Die Implementierung des Guards ist recht simpel. Wenn im localStorage des Browser nicht vorgemerkt wurde, dass die Bedingungen bereits akzeptiert wurde, dann soll auf die entsprechende Seite weitergeleitet werden. Außerdem wird die Rücksprungadresse als Parameter übergeben.

```
export const termsOfUseKey = 'accept-terms-of-use';
export const termsOfUseGuard: CanActivateFn = (route, state) => {
  const accepted = localStorage.getItem(termsOfUseKey);
  if (!Boolean(accepted)) {
    inject(Router).navigate(['terms-of-use'], {
      queryParams: { returnUrl: state.url }
    });
    return false;
  }
  return true;
};
```

Wir ergänzen unsere Routes um die canActivate Eigenschaft wie folgt:

```
{
  path: 'list',
  component: TaskListComponent,
  canActivate: [termsOfUseGuard],
},
{
  path: 'new',
  component: TaskFormComponent,
  canActivate: [termsOfUseGuard],
},
{
  path: 'edit/:id',
  component: TaskFormComponent,
  canActivate: [termsOfUseGuard],
},
{
  path: 'terms-of-use',
  component: TermsOfUseComponent,
},
```

In der Logik der Komponente für die Nutzungsbedingungen müssen wir noch die Persistenz und Weiterleitung implementieren.

```
acceptTerms() {
  localStorage.setItem(termsOfUseKey, 'true');
  const url = this.route.snapshot.queryParams['returnUrl'] ?? '/';
  this.router.navigate([url]);
}
```